

Application of Stack

<u>Infix</u> ^{→ operand}	<u>Post</u>	<u>Prefix</u>
1+2	12+	+12
1-2	12-	-12
2^1	21^	^21

Associativity

2^3^2	2^3^2
8^2	2^9
64	512

() → N/A
 ^ → R to L
 /* → L to R
 +- → L to R

↑ highest
 ↓ lowest

5-2-2	5-3-2
2-2	5-5
0	0

with stack

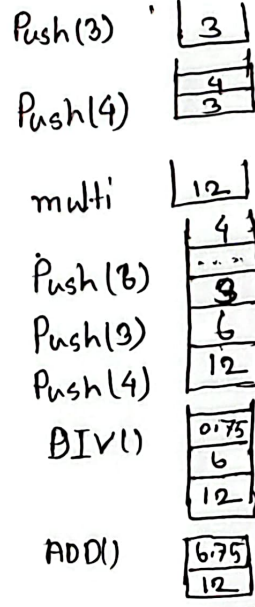
Calculator :

$$\frac{3 \times 4}{6 + \frac{3}{4}}$$

= 1.777...

Inf to Post F

$$\begin{aligned} & (3 \times 4) / (6 + (3 \div 4)) \\ &= (34 \times) / (6 (34 \div) +) \\ &= (34 \times) (6 (34 \div) +) / \\ &= 34 \times 6 34 \div + / \end{aligned}$$



3 ÷ 4

→ DIV() [1.75]

3 condition

① Check stack condition — ①

② Check if scanned character is a bracket — (2,3)

③ if the scanned character is an operator — (4,5,6)

⊗ Q) 2 ^ 3 ^ 4 + 5 * 7

Scanned i/p	Stack	Output
2		2
^	^	2
3	^	2 3
^	^^	2 3
4	^^	2 3 4
+	^	2 3 4 ^
5	+	2 3 4 ^ 5
*	* +	2 3 4 ^ 5
7	* +	2 3 4 ^ 5 7

Power कर 2 लो निज़े ना
इह एता 2 operation इह
इह नाइ निइ यवना निज़े
ना इह पाव

this two

Ans → 234^^57*+

© $1+3 \wedge 2+(5*6-4)*7$

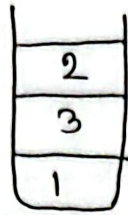
13
* 132

Scanned i/p	stack	output
1		1
+	+	1
3	+	13
^	^+	13
2	^+	132
+	+	132^
+	+	132^+
(	(+	132^+
5	(+	132^+5
*	*(+)	132^+5
6	*(+)	132^+56
-	(+)	132^+56*
-	-(+)	132^+56*
4	-(+)	132^+56*4
)	(+)	132^+56*4-
*	*(+)	132^+56*4-
7	*(+)	132^+56*4-7
		132^+56*4-7*4

'/' ଏବଂ ଅନ୍ୟ
Stack ଡ୍ରାଡ୍ର
ପୋପ୍ ନେଇ
କାଡ୍ର
'/' ଏବଂ output
ପରିଣତ ହୁଏ

* $1321 + 56 \times 4 - 7 \times +$ with stack -

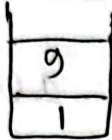
Push(1)



Push(3)

Push(2)

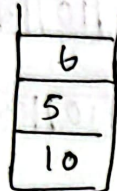
Power(3^2)



Add()

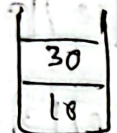


Push(5)

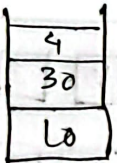


Push(6)

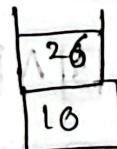
multi



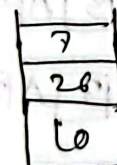
Push(4)



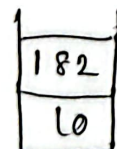
Sub



Push(7)



multi

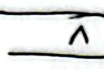
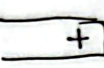
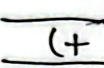
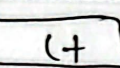
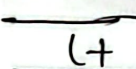
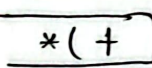
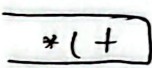
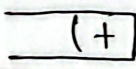


Add()



* 2 or more digit.

$$Q_1 - 10 \wedge 3 + (26 * 6) = 1156$$

%p	Stack	output
1		1
0		10
1		10
3		 
+		  
(		  
2		   
6		    
*		    
6		    
)		     

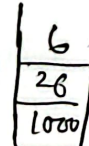
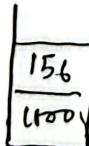
      * + . Ans

now,

Push(10) 
Push(3) 

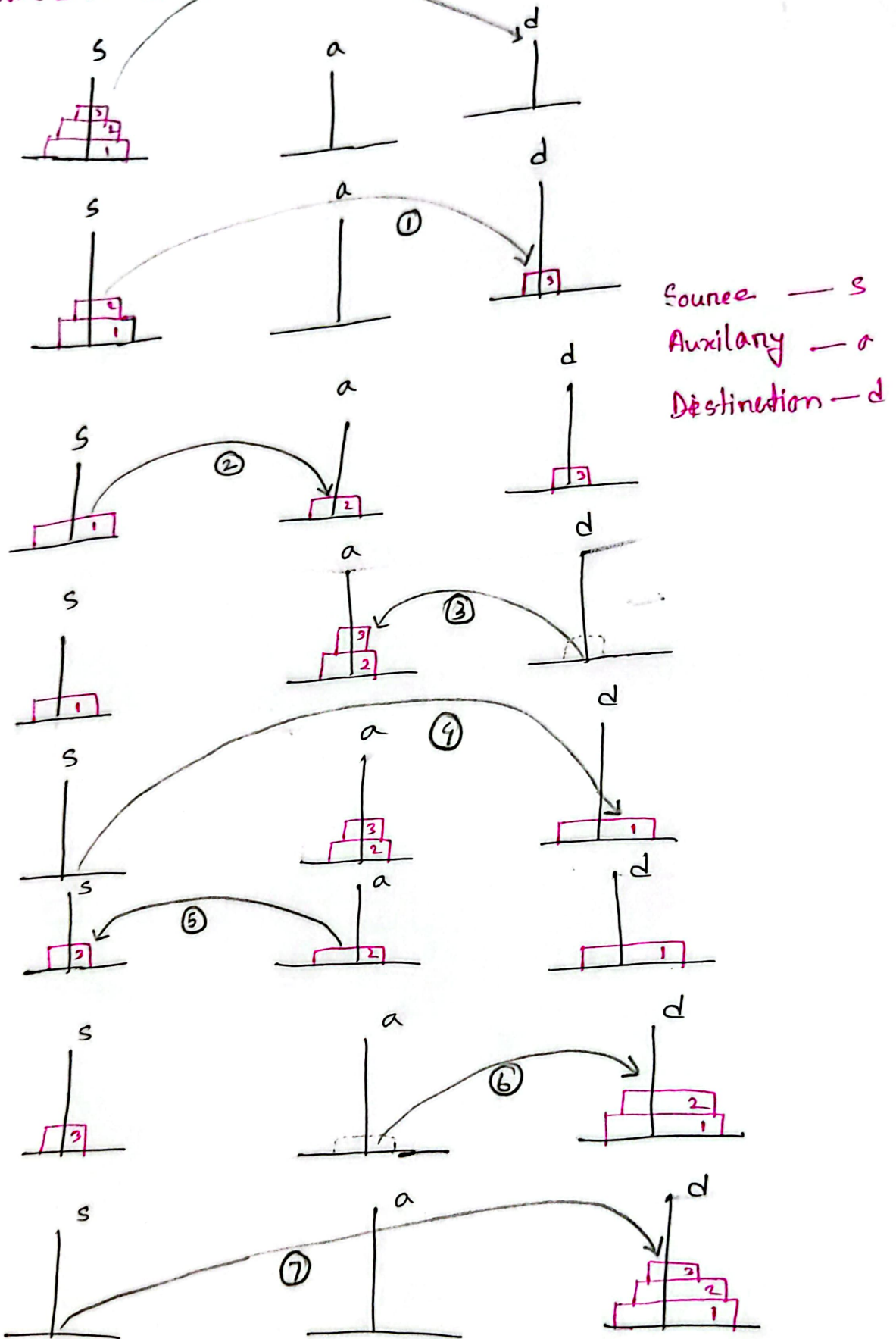
Power() 

Push(26) 

Push(6)  $\rightarrow$ multi()  $\rightarrow$ add()  . Proved.

Recursion

Tower of Hanoi



① ટોર ટોર સરળ રહે

② Pressure હતા નરમ

③ $2^n - 1$ moves રહે

s d o
A B C

| | |
using

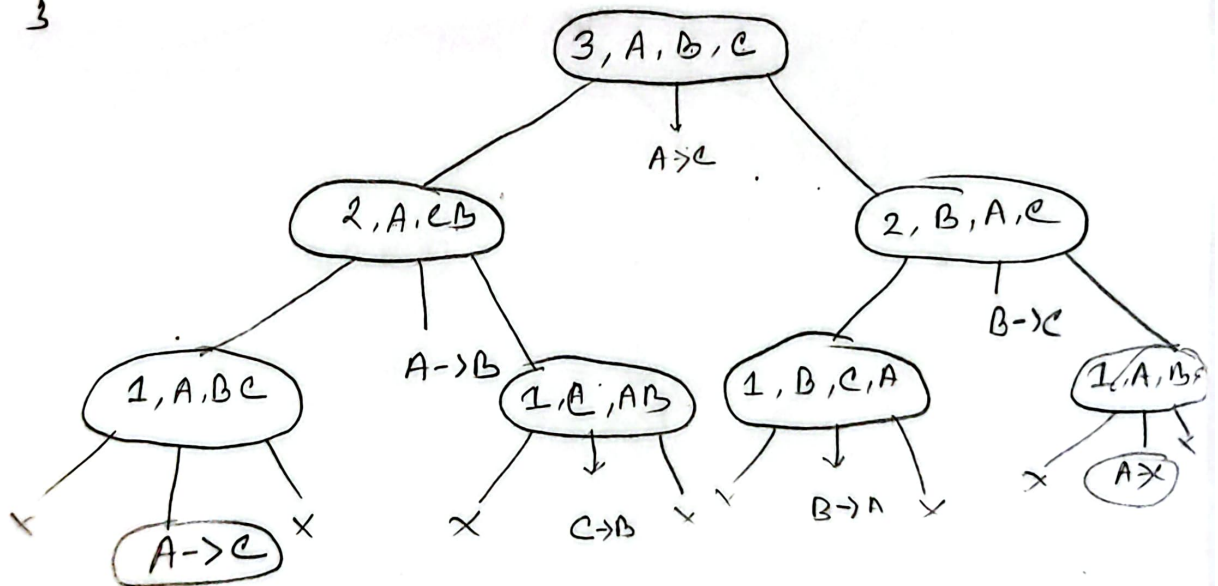
$$A \cong$$
$$B =$$

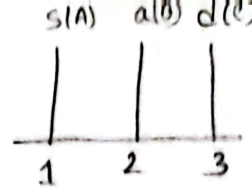
ez

3

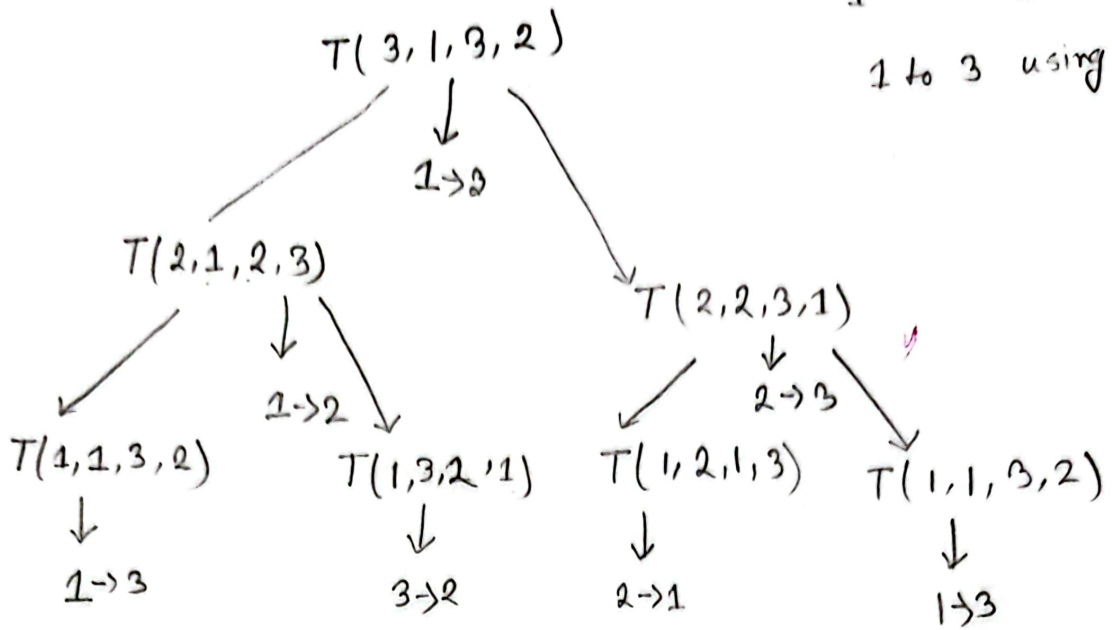
$$x1 = \text{Hanoi}(n-1, s, a, d);$$
$$x_2 = \text{Hanoi}(n-1, a, c, b);$$

3

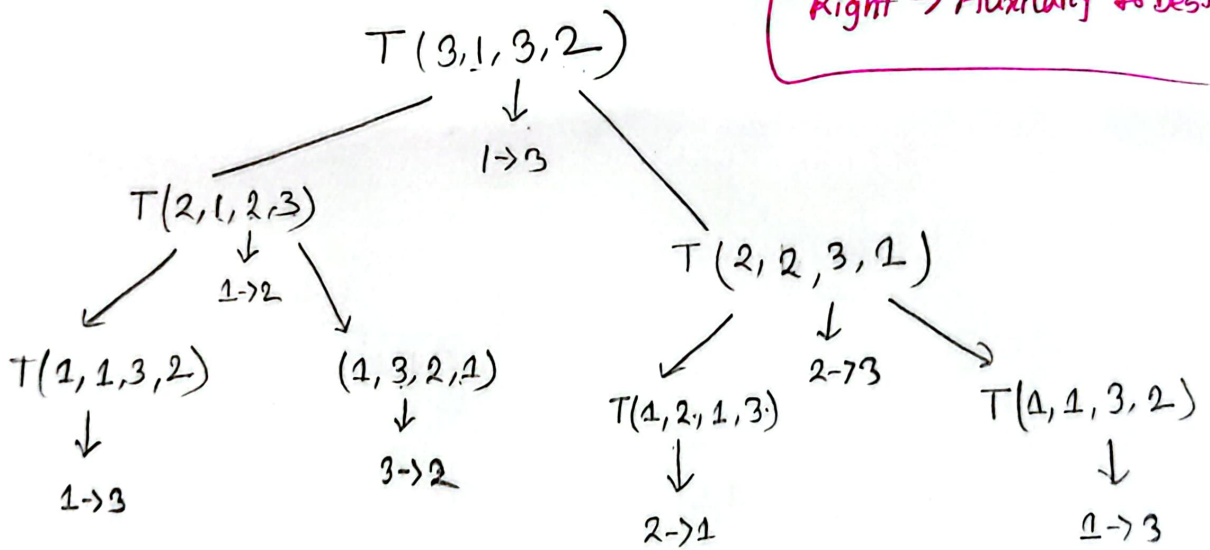




1 to 3 using 2



Left → Source to Aux
 Right → Auxiliary to Dest



$1 \rightarrow 3$ $1 \rightarrow 2$ $3 \rightarrow 2$ $1 \rightarrow 3$ $2 \rightarrow 1$ $2 \rightarrow 3$ $1 \rightarrow 3$

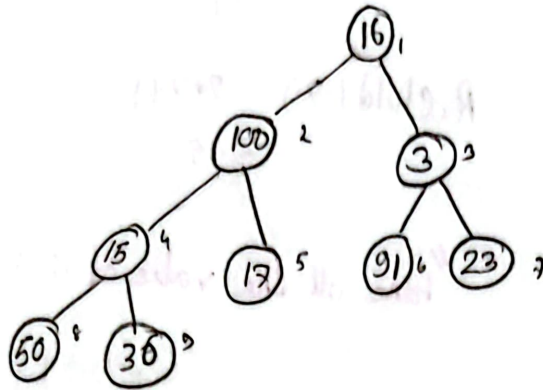
Binary Heap

	16	100	3	15	17	91	23	50	36
0	1	2	3	4	5	6	7	8	9

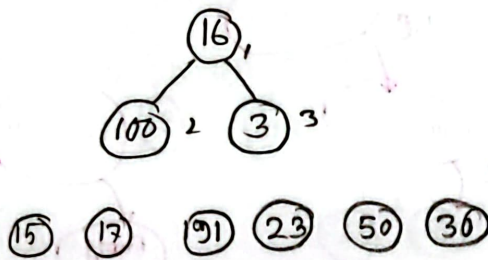
length = 9 [indexing $\rightarrow 1-9$]

heap size = 9

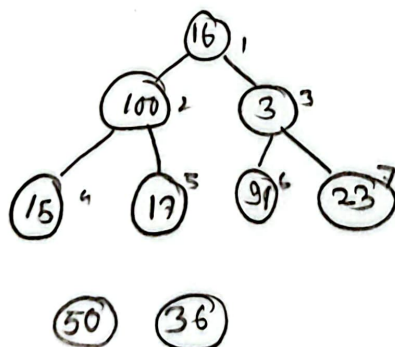
heap size ka value khatam
element serials ka khatam hai



heap size (9) (-)



heap size (7) (+)



Given array $\rightarrow$ Parent (0/1) Index is 0/1, child $\rightarrow$ left, right
 # Who is the parent of 7th element?

Parent (i) = $\left\lfloor \frac{i}{2} \right\rfloor$

Parent (7) = $\left\lfloor \frac{7}{2} \right\rfloor$
 = $\left\lfloor 3.5 \right\rfloor$
 = 3.

① L.child (7) = 2×7 left child $\leq$ heap size
 = 14

R.child (7) = $2 \times 7 + 1$
 = 15

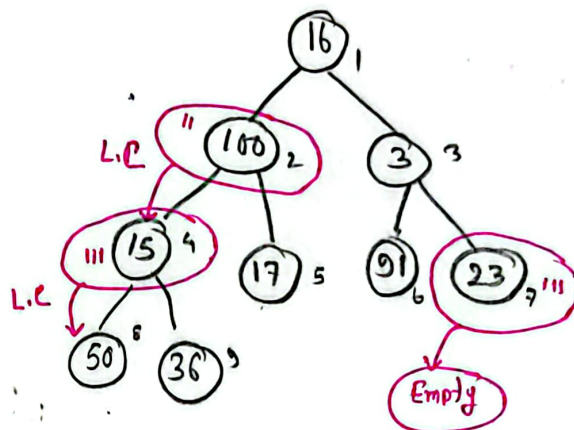
"here all the value of (i) is index value"

② left child (i) = $2 \times i$

left child (2) = $2 \times 2 = 4$

Right child (i) = $2i + 1$

Right child (2) = $2 \times 2 + 1$
 = 5



③ left child (4) = $2 \times 4 = 8$

Max heap

Parent $\geq$ Children [Parent Child $\geq$ 200 200 Max heap]

Min heap

16 14 10 8 7 9 3 2 1 4
P

Parent $\leq$ Children

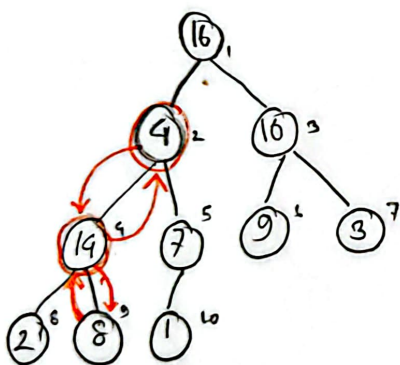
3 4 10 8 7 19 13 12 14 16
P

Max Heapify ($O(\log n)$)

→ try to make array more

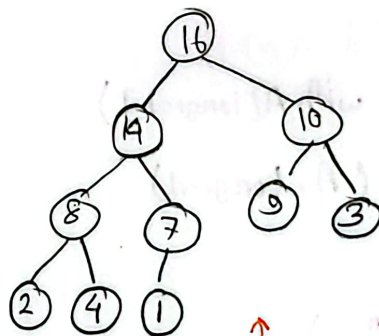
Build Max Heapify ($O(n)$)

→ Convert an array in to a heap.



maxHeapify(A, 2)

maxHeapify(A, 4)



after calling maxHeapify() Function

code

maxHeapify(A, i)

left = 2*i;

Right = 2*i + 1;

if (left ≤ heapSize && A[i] > A[left])

largest = i;

else

largest = left

if (Right ≤ heapSize && A[Right] > A[largest])

largest = right

if i != largest

swap(A[i] with A[largest])

maxHeapify(A, largest)

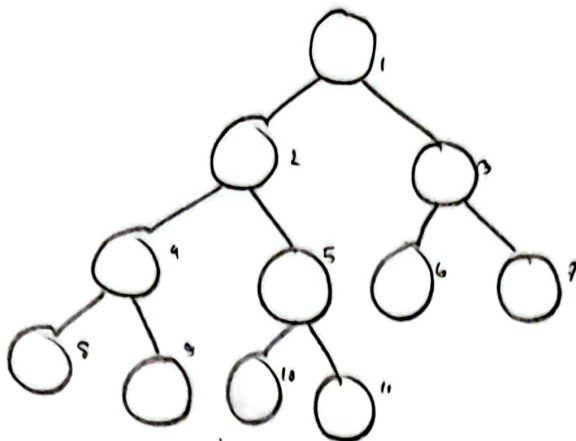
void MaxHeapify(A)

A.heapSize = A.length

for i = $\left\lfloor \frac{A.length}{2} \right\rfloor$ down to 1

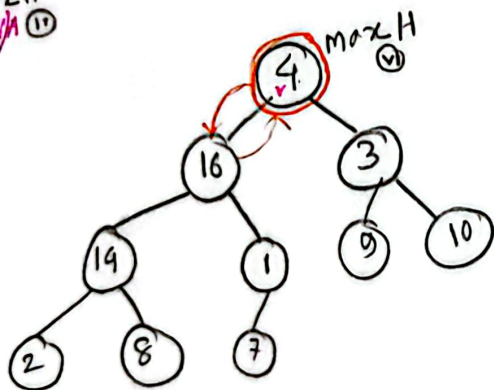
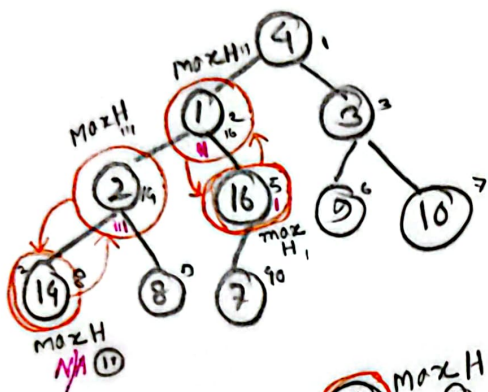
maxHeapify(A, i)

Build Max Heapify $\rightarrow$ Convert an array into a Heap.



last index તો 2 થી parent
 તારા તરફ ceiling value નો લેવામાં
 જે કે parent કે children માંથી
 જે index હોય main root બની maxheapify
 function call થાય

$$A(i) = \left\lfloor \frac{10}{2} \right\rfloor = \left\lfloor 5 \right\rfloor = 5$$



ct-3
Heap Sort

Heap Sort (A)

1. Build-Max-Heapify (A)
2. for $i = A.length \cdot \text{down to } 2$)
3. swap $A[1]$ with $A[i]$
4. $A.heapSize = A.heapSize - 1$
5. MaxHeapify (A, 1)

16, 14, 10, 8, 7, 9, 3, 2, 4, 1

